
DATA CUBES

E0 261

Jayant Haritsa
Indian Institute of Science



Introduction

- Organizations analyze historical data to identify useful patterns and support business strategies.
- Emphasis is on complex, interactive, exploratory analysis of very large datasets created by integrating data from across all parts of an enterprise; data is fairly static.
 - Contrast such **On-Line Analytic Processing (OLAP)** with **On-line Transaction Processing (OLTP)**: mostly long queries, instead of short update Xacts.



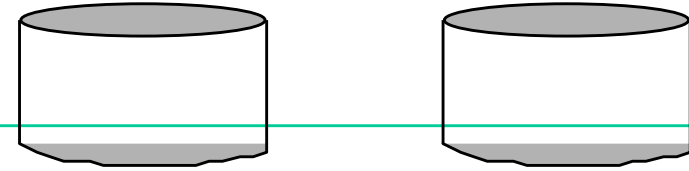
Overview

- **Data Warehousing**: Consolidate data from many sources in one large repository
 - Loading, periodic synchronization of replicas.
 - Semantic integration.
- **OLAP**:
 - Complex SQL queries and views.
 - Queries based on spreadsheet-style operations and “multidimensional” view of data.
 - Interactive and “online” queries.
- Note that Data Warehouses form the substrate on which **Data Mining** can be carried out.

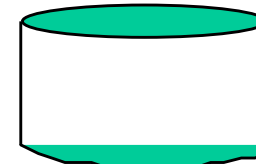


Data Warehousing

EXTERNAL DATA SOURCES



**EXTRACT
TRANSFORM
LOAD
REFRESH**

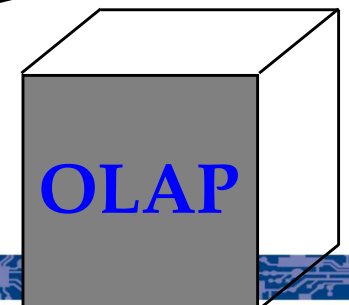


**Metadata
Repository**



**DATA
WAREHOUSE**

SUPPORTS



- Integrated data spanning long time periods, often augmented with summary information.
- Several terabytes to petabytes common.
- Interactive response times expected for complex queries; ad-hoc updates uncommon.

Warehousing Issues

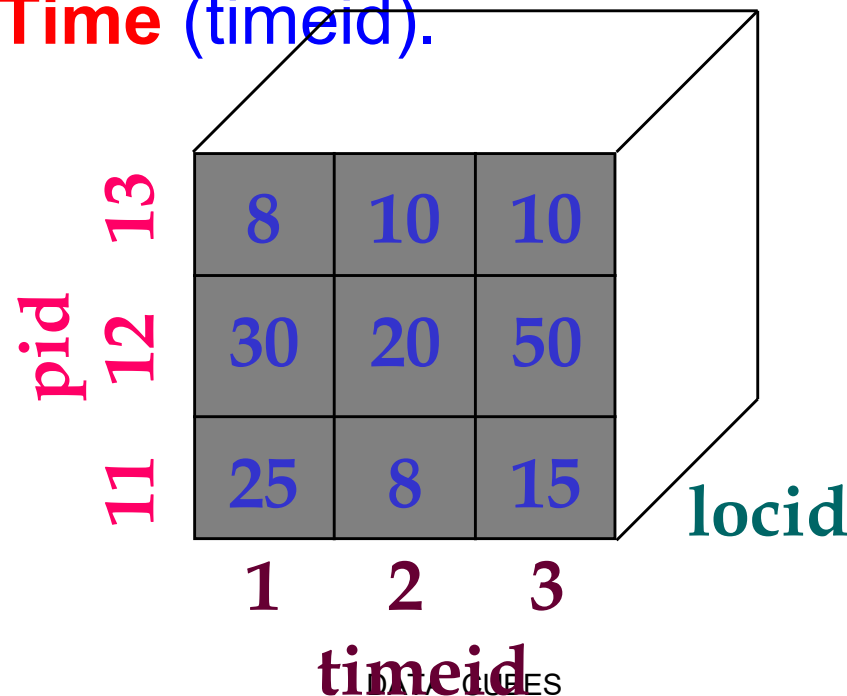
- Semantic Integration: When getting data from multiple sources, must eliminate mismatches, e.g., different currencies, schemas.
- Heterogeneous Sources: Must access data from a variety of source formats and repositories.
- Load, Refresh, Purge: Must load data, periodically refresh it, and purge too-old data.
- Metadata Management: Must keep track of source, loading time, and other information for all data in the warehouse.



Multidimensional Data Model

- Collection of numeric measures, which depend on a set of dimensions.
 - e.g. measure **Sales**, dimensions **Product** (pid), **Location** (locid), and **Time** (timeid).

Slice locid=1 is shown:



pid	timeid	locid	sales
11	1	1	25
11	2	1	8
11	3	1	15
12	1	1	30
12	2	1	20
12	3	1	50
13	1	1	8
13	2	1	10
13	3	1	10
11	1	2	35

MOLAP vs ROLAP

- Multidimensional data can be stored physically in a (disk-resident, persistent) array; called **MOLAP** systems (covered in TIDS course)
 - Essbase
- Alternatively, can store as a relation; called **ROLAP** systems (today's paper)
 - Redbrick, Oracle



ROLAP Tables

- The main relation, which relates dimensions to a measure, is called the **fact table**.
 - Example: **SALES(pid, timeid, locid, sales)**
- Each dimension can have additional attributes and an associated **dimension table**.
 - Example: **Products(pid, pname, category, price)**
- Fact tables are **much** larger than dimensional tables. Their schema is **{foreign keys of all dimension tables + all measure attributes}**



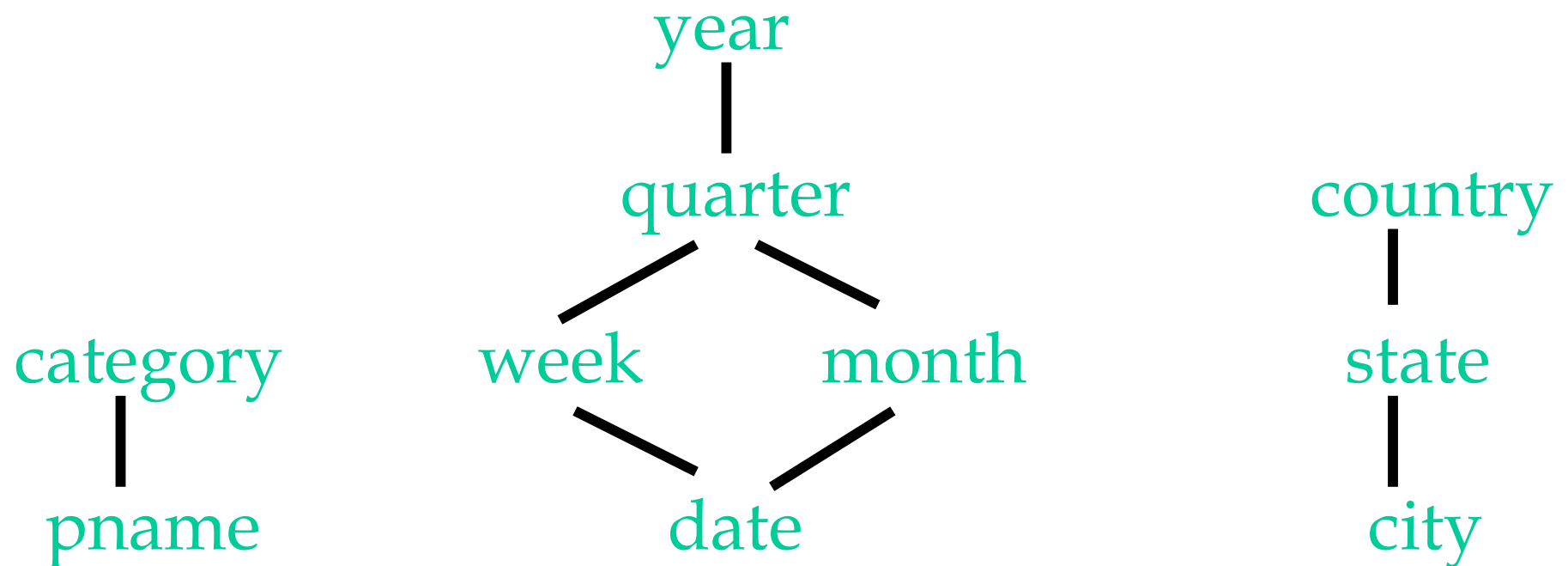
Dimension Hierarchies

- For each dimension, the set of values can be organized in a hierarchy:

PRODUCT

TIME

LOCATION



OLAP Queries

- Influenced by SQL and by spreadsheets.
- A common operation is to **aggregate** a measure over one or more dimensions.
 - Find total sales.
 - Find total sales for each city, or for each state.
 - Find top five products ranked by total sales.
- **Roll-up:** Aggregating at different levels of a dimension hierarchy.
 - E.g., Given total sales by city, we can roll-up to get sales by state.



OLAP Queries (contd)

- **Drill-down:** The inverse of roll-up.
 - e.g. Given total sales by state, can drill-down to get total sales by city.
 - e.g. Can also drill-down on different dimension to get total sales by product for each state.
- **Pivoting:** Aggregation on selected dimensions.
 - e.g. Pivoting on Location and Time yields this **cross-tabulation**:
 - equivalent to “rotation”

	WB	UP	Total
1995	63	81	144
1996	38	107	145
1997	75	35	110
Total	176	223	339

OLAP Queries (contd)

- **Slicing** : Equality selections on one or more dimensions.
- **Dicing**: Range selections on one or more dimensions.



Comparison with SQL Queries

- The cross-tabulation obtained by pivoting can also be computed using a collection of SQL queries:

```
SELECT SUM(S.sales)
FROM   Sales S, Times T, Locations L
WHERE  S.timeid=T.timeid AND S.locid=L.locid
GROUP BY T.year, L.state
```

```
SELECT SUM(S.sales)
FROM   Sales S, Times T
WHERE  S.timeid=T.timeid
GROUP BY T.year
```

```
SELECT SUM(S.sales)
FROM   Sales S, Location L
WHERE  S.locid=L.locid
GROUP BY L.state
```



3-D CUBE Model

Aggregate



Sum

Group By (with total)

By Color

RED
WHITE
BLUE



Sum

Cross Tab

Chevy Ford By Color

RED
WHITE
BLUE

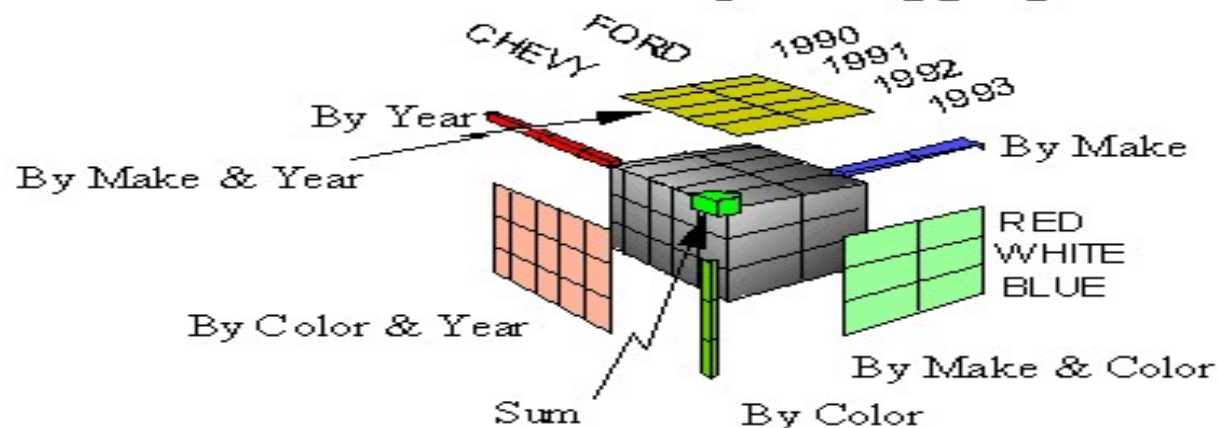


By Make



Sum

The Data Cube and The Sub-Space Aggregates



The CUBE Operator

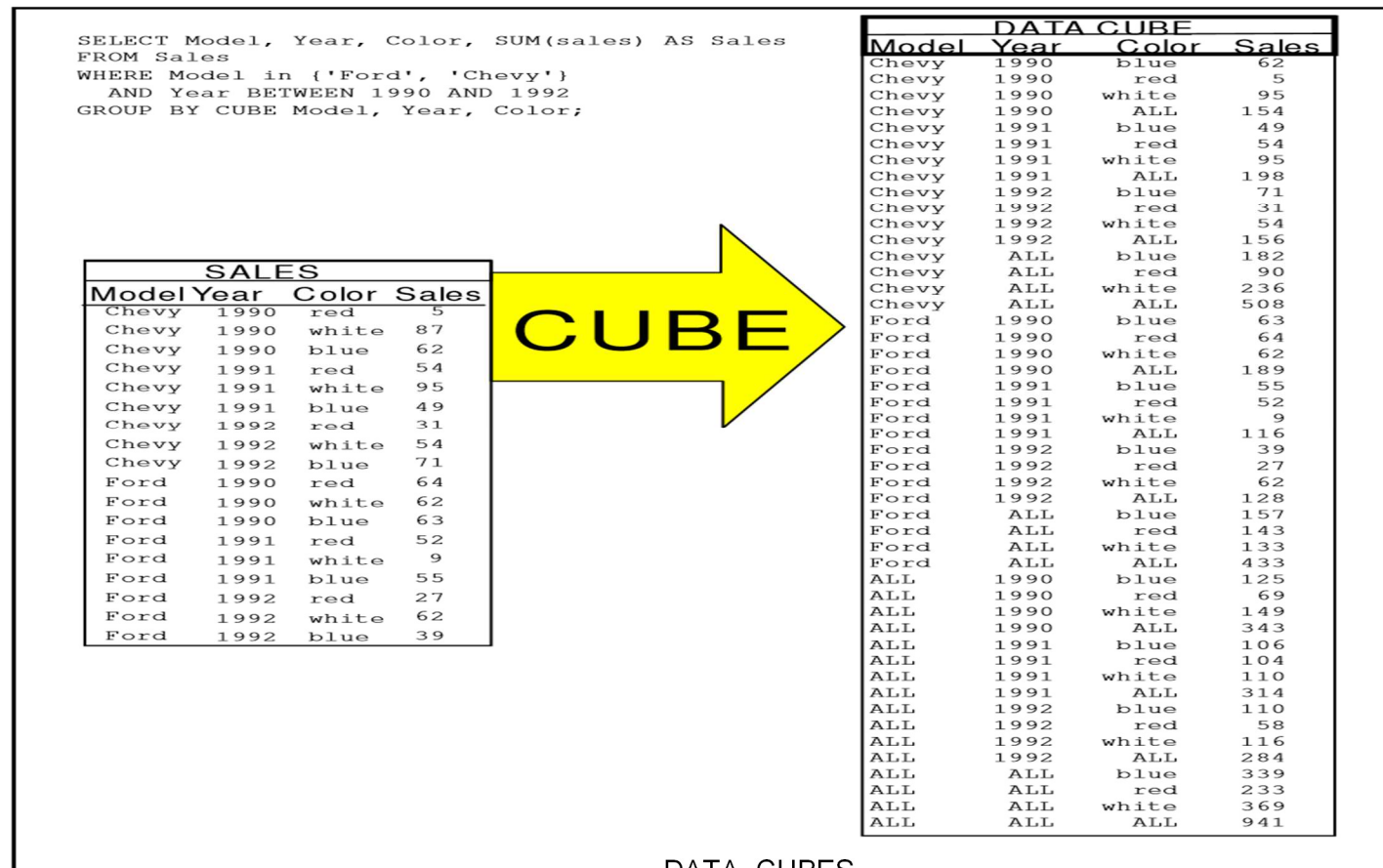
- Generalizing the previous example, if there are k dimensions, we have 2^k possible SQL GROUP BY queries that can be generated through pivoting on a subset of dimensions.
- CUBE pid, locid, timeid BY SUM Sales
 - Equivalent to rolling up Sales on all eight subsets of the set {pid, locid, timeid}; each roll-up corresponds to an SQL query of the form:

Lots of work on
optimizing the CUBE operator!

```
SELECT SUM(S.sales)
FROM   Sales S
GROUP BY grouping-list
```

CUBE TABLES

- Use “ALL” to represent the set over which aggregation is computed (Fig 4)



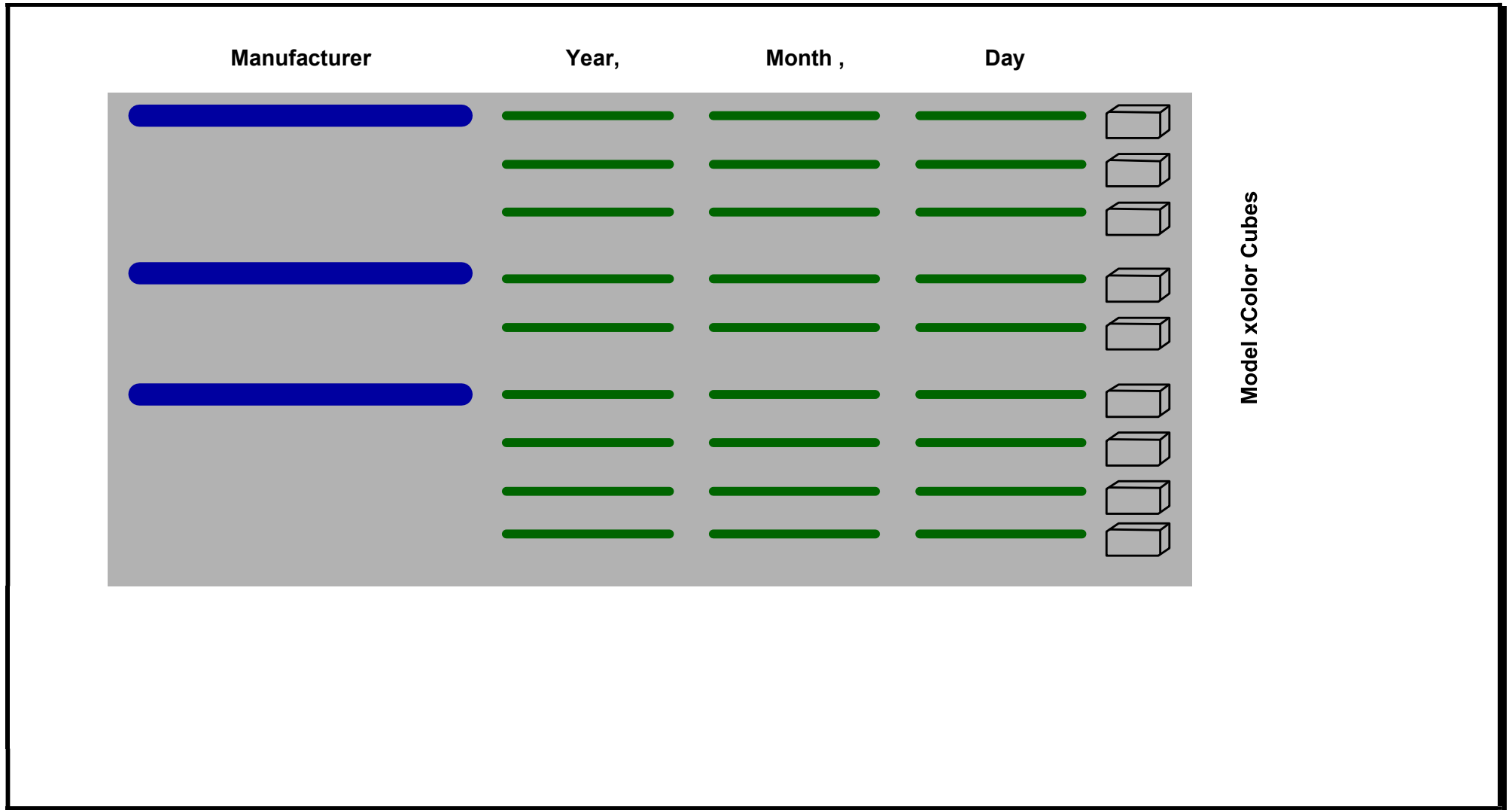
Aggregation Hierarchy

- GROUP BY <select list 3>
 ROLLUP <select list 2>
 CUBE <select list 1>

```
SELECT manufacturer, year, month, day, color, model, SUM (price) as revenue
FROM Sales
GROUP BY    Manufacturer
           ROLLUP    Year(Time) as Year, Month(Time) as Month, Day(Time) as Day,
           CUBE      Color, Model
```



Figure 5



CUBE Research Issues

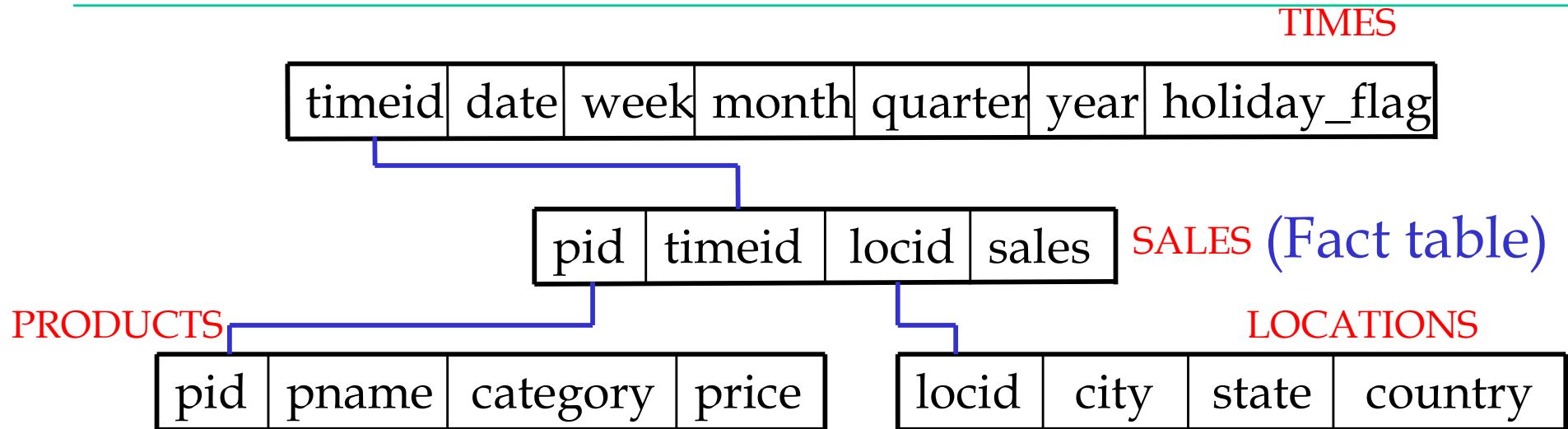
- Storage Structures for Cube (ROLAP, MOLAP)
- Schemas for ROLAP Cube (Star, Snowflake)
- Index Structures for Cubes
- Level of Materialization of Cube (next paper)
-



CUBE SCHEMAS

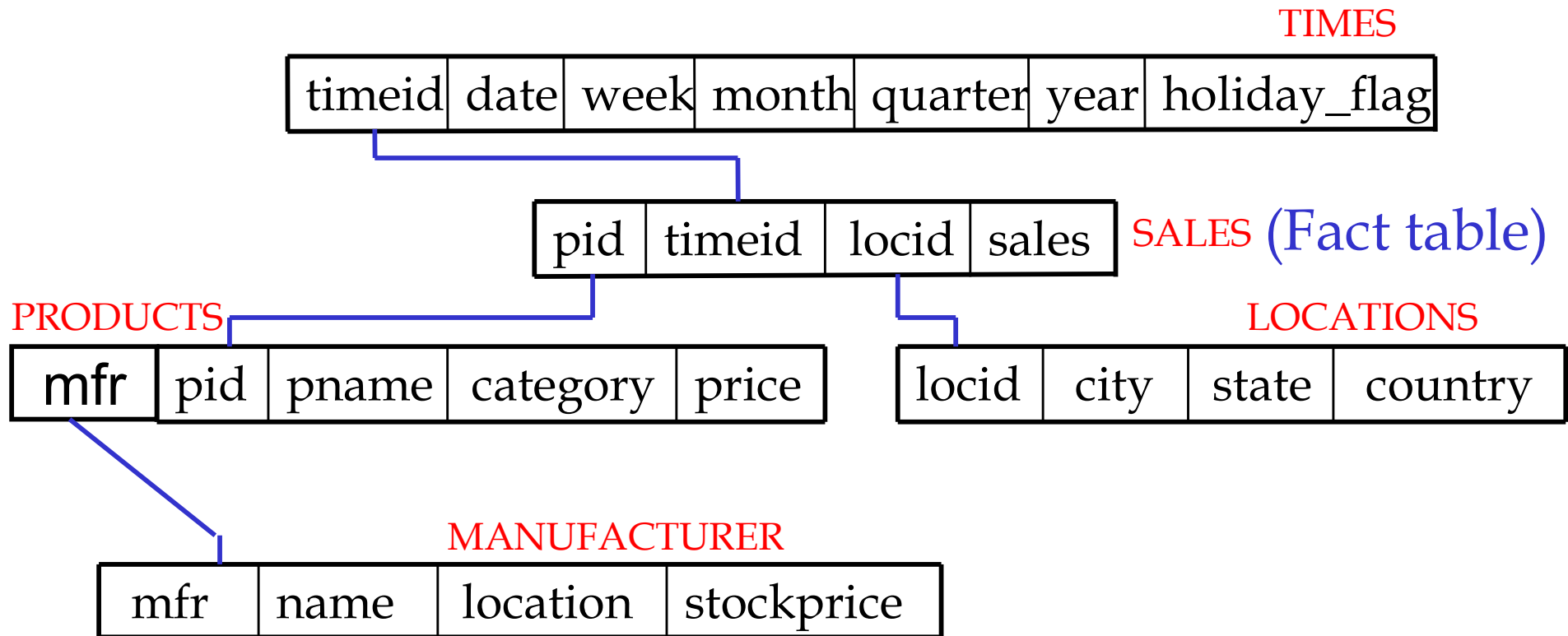


Star Schema



- Fact table in BCNF; dimension tables not normalized.
 - Dimension tables are small; updates/inserts/deletes are rare. So, anomalies less important than good query performance.
- Computing join of all these relations is called a **star join**.

Snowflake Schema



- Dimension tables also normalized to reduce redundancy.

CUBE INDEXES



Indexes

- Joins common between fact table and dimension tables
- Interactive response is expected, so indexes are required
- Problems:
 - Space
 - maintenance not a problem because read-only data

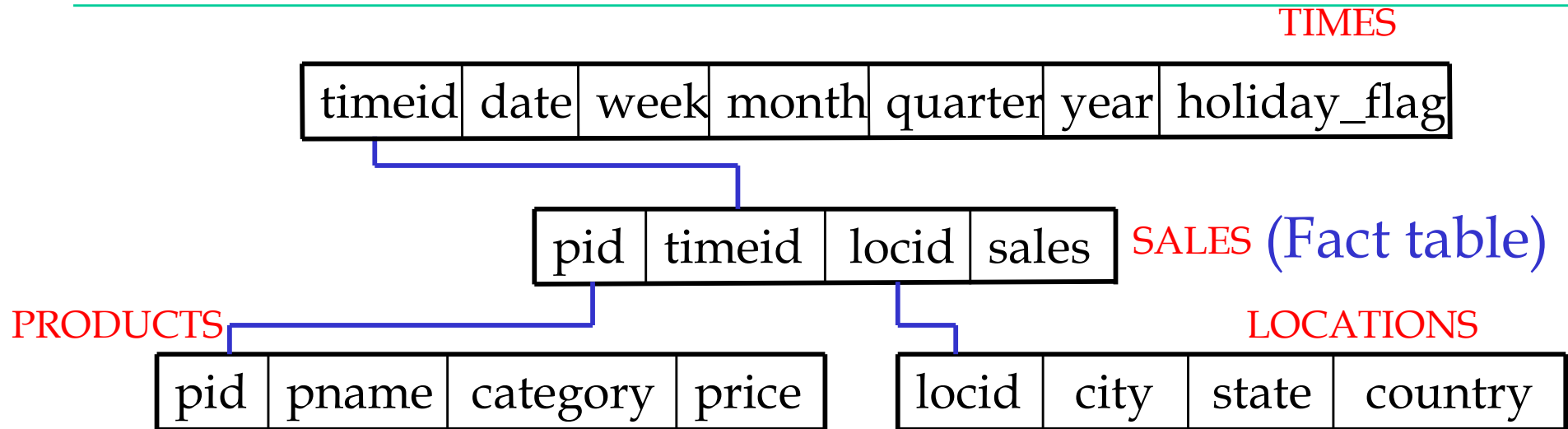


Join Indexes

- Consider join of Sales, Products, Times, and Locations, possibly with selection conditions (e.g. country="India").
 - A **join index** can be constructed to speed up such joins. The index contains **[p,t,l,s]** if there are tuples **p** in Products, **t** in Times and **l** in Locations that satisfy the join (with sid) **s** in Sales .
e.g. [Beer, Oct 11, Malleswaram, {2567,3089,156323}]
- Problem: Number of join indexes can grow rapidly.
 - A variant of the idea addresses this problem: For each column with an additional selection (e.g. country), build an index with **[c,s]** in it if a dimension table tuple with value **c** in the selection column joins with a Sales tuple with sid **s**; if indexes are bitmaps, called **bitmapped join index**.



Bitmapped Join Index



- Consider a query with conditions **price=10** and **country="India"**. Suppose tuple (with sid) **s** in Sales joins with tuple **p** with price=10 and tuple **l** with country = "India". Two join indexes; one has **[10,s]** and the other **[India,s]**.
- Intersecting these indexes tells us which tuples in Sales are in the join and satisfy the given selection.

Summary

- Decision support is a major domain of databases.
- Involves the creation of large, consolidated data repositories called data warehouses.
- Warehouses exploited using sophisticated analysis techniques: complex SQL queries and OLAP “multidimensional” queries (influenced by both SQL and spreadsheets).
- New techniques for database design, indexing, view maintenance, and interactive querying need to be supported.

END DATA CUBES

E0 261

